

CO2-LAB EXPERIMENTS

1. Map Coloring to implement CSP.

Program:

```
# Map Coloring using CSP
```

```
colors = ["Red", "Green", "Blue"]
```

```
# Map connections
```

```
graph = {
```

```
    "A": ["B", "C"],
```

```
    "B": ["A", "C", "D"],
```

```
    "C": ["A", "B", "D"],
```

```
    "D": ["B", "C"]
```

```
}
```

```
color = {}
```

```
def is_safe(region, c):
```

```
    for neighbor in graph[region]:
```

```
        if neighbor in color and color[neighbor] == c:
```

```
            return False
```

```
    return True
```

```
def solve(regions):
```

```
    if len(color) == len(regions):
```

```
        return True
```

```
    region = regions[len(color)]
```

```
    for c in colors:
        if is_safe(region, c):
            color[region] = c

        if solve(regions):
            return True

        del color[region]

    return False
regions = list(graph.keys())

if solve(regions):
    print("Map Coloring Solution:")
    for region in regions:
        print(region, "->", color[region])
else:
    print("No solution exists")
```

Output:

```
-----
Map Coloring Solution:
A -> Red
B -> Green
C -> Blue
D -> Red
|
```

2. TIC TAC TOE

Program

```
# Tic Tac Toe Game
```

```
board = [" " for _ in range(9)]
```

```
def display():
```

```
    print()
    print(board[0], "|", board[1], "|", board[2])
    print("--+---+--")
    print(board[3], "|", board[4], "|", board[5])
    print("--+---+--")
    print(board[6], "|", board[7], "|", board[8])
    print()
```

```
def check_winner(player):
```

```
    win = [
        (0, 1, 2), (3, 4, 5), (6, 7, 8), # rows
        (0, 3, 6), (1, 4, 7), (2, 5, 8), # columns
        (0, 4, 8), (2, 4, 6)             # diagonals
    ]
    for a, b, c in win:
        if board[a] == board[b] == board[c] == player:
            return True
    return False
```

```
player = "X"
```

```
for turn in range(9):
```

```
    display()
```

```
    pos = int(input(f"Player {player}, Enter position (1-9): ")) - 1
```

```
    if board[pos] != " ":
```

```
print("Position already filled! Try again.")  
continue
```

```
board[pos] = player
```

```
if check_winner(player):  
    display()  
    print("Player", player, "Wins!")  
    break
```

```
# Switch player  
player = "O" if player == "X" else "X"
```

```
else:  
    display()  
    print("Match Draw!")
```

Output:

```
  |  |
--+---+--
  |  |
--+---+--
  |  |
```

Player X, Enter position (1-9): 1

```
X |  |
--+---+--
  |  |
--+---+--
  |  |
```

Player O, Enter position (1-9): 3

```
X |  | O
--+---+--
  |  |
--+---+--
  |  |
```

Player X, Enter position (1-9): 4

```
X |  | O
--+---+--
X |  |
--+---+--
  |  |
```

Player O, Enter position (1-9): 7

```
X |  | O
--+---+--
X |  |
--+---+--
O |  |
```

Player X, Enter position (1-9): 5

```
X |  | O
--+---+--
X | X |
--+---+--
O |  |
```

Player O, Enter position (1-9): 9

```
X |  | O
--+---+--
X | X |
--+---+--
O |  | O
```

Player X, Enter position (1-9): 6

```
X |  | O
--+---+--
X | X | X
--+---+--
O |  | O
```

Player X Wins!

3. Minimax algorithm for gaming

Program:

Minimax Algorithm with User Input

```
def minimax(values, isMax):
```

```
    # Base case
```

```

if len(values) == 1:
    return values[0]

temp = []
# Compare values in pairs
for i in range(0, len(values), 2):
    if isMax:
        temp.append(max(values[i], values[i + 1]))
    else:
        temp.append(min(values[i], values[i + 1]))

```

```

# Display current level
if isMax:
    print("Max Level :", temp)
else:
    print("Min Level :", temp)

```

```

# Recursive call
return minimax(temp, not isMax)

```

```

# Main Program
n = int(input("Enter number of leaf nodes (Power of 2): "))
values = []

```

```

print("Enter the leaf node values:")
for i in range(n):
    value = int(input(f"Value {i+1}: "))
    values.append(value)

```

```

print("\nLeaf Nodes:", values)

```

```

answer = minimax(values, True)
print("\nOptimal Value =", answer)

```

Output:

```
Enter number of leaf nodes (Power of 2): 8
Enter the leaf node values:
Value 1: 6
Value 2: 8
Value 3: 2
Value 4: 4
Value 5: 6
Value 6: 2
Value 7: 7
Value 8: 9

Leaf Nodes: [6, 8, 2, 4, 6, 2, 7, 9]
Max Level : [8, 4, 6, 9]
Min Level : [4, 6]
Max Level : [6]

Optimal Value = 6
```

4. Apha & Beta pruning algorithm for gaming

Program

Alpha-Beta Pruning for Gaming

```
import math
```

```
def alpha_beta(depth, node, maximizing, values, alpha, beta):
```

```
    # Leaf node
```

```
    if depth == 0:
```

```
        return values[node]
```

```
    if maximizing:
```

```
        best = -math.inf
```

```
    for i in range(2):
```

```
        value = alpha_beta(depth - 1, node * 2 + i,
                           False, values, alpha, beta)
```

```
best = max(best, value)
alpha = max(alpha, best)
```

```
# Alpha-Beta pruning
if beta <= alpha:
    break
```

```
return best
```

```
else:
```

```
best = math.inf
```

```
for i in range(2):
```

```
    value = alpha_beta(depth - 1, node * 2 + i,
                        True, values, alpha, beta)
```

```
best = min(best, value)
```

```
beta = min(beta, best)
```

```
# Alpha-Beta pruning
```

```
if beta <= alpha:
```

```
    break
```

```
return best
```

```
# Main Program
```

```
depth = int(input("Enter depth of game tree: "))
```

```
nodes = 2 ** depth
```

```
values = []
```

```
print("Enter", nodes, "leaf node values:")
```



```

for i in range(nodes):
    value = int(input("Leaf " + str(i + 1) + ": "))
    values.append(value)

result = alpha_beta(
    depth, 0, True, values,
    -math.inf, math.inf
)

print("\nBest value for MAX player:", result)

```

Output:

```

Enter depth of game tree: 4
Enter 16 leaf node values:
Leaf 1: 2
Leaf 2: 5
Leaf 3: 9
Leaf 4: 3
Leaf 5: 12
Leaf 6: 16
Leaf 7: 8
Leaf 8: 9
Leaf 9: 19
Leaf 10: 22
Leaf 11: 14
Leaf 12: 32
Leaf 13: 75
Leaf 14: 29
Leaf 15: 13
Leaf 16: 65

Best value for MAX player: 19

```

5. Decision Tree

Program:

Alpha-Beta Pruning for Gaming

```
import math
```

```
def alpha_beta(depth, node, maximizing, values, alpha, beta):
```

```
    # Leaf node
```

```
    if depth == 0:
```

```
        return values[node]
```

```
    if maximizing:
```

```
        best = -math.inf
```

```
    for i in range(2):
```

```
        value = alpha_beta(depth - 1, node * 2 + i,  
                            False, values, alpha, beta)
```

```
        best = max(best, value)
```

```
        alpha = max(alpha, best)
```

```
    # Alpha-Beta pruning
```

```
    if beta <= alpha:
```

```
        break
```

```
    return best
```

```
else:
```

```
    best = math.inf
```

```
    for i in range(2):
```

```
        value = alpha_beta(depth - 1, node * 2 + i,  
                            True, values, alpha, beta)
```

```
        best = min(best, value)
```

```
        beta = min(beta, best)
```

```
    # Alpha-Beta pruning
```

```
    if beta <= alpha:
```

```
        break
```

```

        return best

# Main Program

depth = int(input("Enter depth of game tree: "))

nodes = 2 ** depth
values = []

print("Enter", nodes, "leaf node values:")

for i in range(nodes):
    value = int(input("Leaf " + str(i + 1) + ": "))
    values.append(value)

result = alpha_beta(
    depth, 0, True, values,
    -math.inf, math.inf
)

print("\nBest value for MAX player:", result)

```

Output:

```

Enter depth of game tree: 2
Enter 4 leaf node values:
Leaf 1: 3
Leaf 2: 5
Leaf 3: 4
Leaf 4: 9

Best value for MAX player: 4
|

```

6. Feed forward neural Network

Program:

```
import math

# Sigmoid activation function
def sigmoid(x):
    return 1 / (1 + math.exp(-x))

# Get input from user
print("Enter two input values:")
x1 = float(input("X1: "))
x2 = float(input("X2: "))

print("Enter target output:")
target = float(input("Target: "))

# Initial weights
w1 = 0.5
w2 = 0.5
w3 = 0.5
w4 = 0.5

learning_rate = 0.5

# ----- Forward Pass -----

# Hidden layer
h1 = sigmoid(x1 * w1 + x2 * w2)
h2 = sigmoid(x1 * w3 + x2 * w4)

# Output layer
output = sigmoid(h1 + h2)

print("\nOutput before training:", round(output, 4))
```

```

# ----- Backpropagation -----

error = target - output

# Output layer delta
delta_output = error * output * (1 - output)

# Update weights
w1 = w1 + learning_rate * delta_output * x1
w2 = w2 + learning_rate * delta_output * x2
w3 = w3 + learning_rate * delta_output * x1
w4 = w4 + learning_rate * delta_output * x2

# ----- Forward Pass Again -----

h1 = sigmoid(x1 * w1 + x2 * w2)
h2 = sigmoid(x1 * w3 + x2 * w4)

output = sigmoid(h1 + h2)

print("Output after training:", round(output, 4))
print("Error:", round(target - output, 4))

```

Output:

```

Enter two input values:
X1: 5
X2: 3
Enter target output:
Target: 2

Output before training: 0.877
Output after training: 0.8803
Error: 1.1197

```

7. Prolog Program to Sum the Integers from 1 to n.

Program:

```
sum(0, 0).
```

```
sum(N, S) :-
```

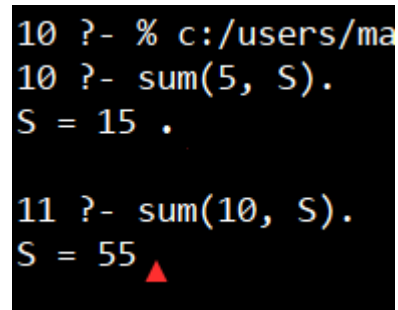
```
    N > 0,
```

```
    N1 is N - 1,
```

```
    sum(N1, S1),
```

```
    S is N + S1.
```

Output:



```
10 ?- % c:/users/ma
10 ?- sum(5, S).
S = 15 .

11 ?- sum(10, S).
S = 55 ▲
```

8. Prolog Program for Database with NAME and DOB

Program:

```
% Name and DOB Database
```

```
person(rahul, '10-05-2004').
```

```
person(priya, '15-08-2003').
```

```
person(arun, '20-01-2005').
```

```
person(sneha, '25-12-2004').
```

```
get_dob(Name, DOB) :-
```

```
    person(Name, DOB).
```

```
get_name(DOB, Name) :-
```

```
    person(Name, DOB).
```

Output:

```
[1] 18 ?- dob(rahul, DOB).  
DOB = '10-05-2004'.  
  
[1] 19 ?- name('15-08-2003', Name).  
Name = priya.  
  
[1] 20 ?- person(Name, DOB).  
Name = rahul,  
DOB = '10-05-2004' .
```

9. Prolog Program for STUDENT-TEACHER-SUB-CODE

Program:

% Student - Teacher - Subject Code Database

student_teacher_subject(rahul, kumar, cs101).

student_teacher_subject(priya, anitha, cs102).

student_teacher_subject(arun, kumar, cs103).

student_teacher_subject(sneha, anitha, cs104).

% Find teacher and subject using student

find_student(Student, Teacher, Subject) :-

student_teacher_subject(Student, Teacher, Subject).

% Find students using teacher

find_teacher(Teacher, Student, Subject) :-

student_teacher_subject(Student, Teacher, Subject).

% Find student and teacher using subject code

find_subject(Subject, Student, Teacher) :-

student_teacher_subject(Student, Teacher, Subject).

Output:

```
[1] 22 ?- find_student(rahul, Teacher, Subject).
Teacher = kumar,
Subject = cs101.

[1] 23 ?- find_teacher(kumar, Student, Subject).
Student = rahul,
Subject = cs101 .

[1] 24 ?- find_subject(cs102, Student, Teacher).
Student = priya,
Teacher = anitha.
```

10. Prolog Program for PLANETS Database

Program:

% Planets Database

planet(mercury, 1).

planet(venus, 2).

planet(earth, 3).

planet(mars, 4).

planet(jupiter, 5).

planet(saturn, 6).

planet(uranus, 7).

planet(neptune, 8).

% Find position using planet name

find_position(Planet, Position) :-

planet(Planet, Position).

% Find planet using position

find_planet(Position, Planet) :-

planet(Planet, Position).

Output:

```
[1] 28 ?- find_position(earth, Position).  
Position = 3.  
  
[1] 29 ?- find_planet(4, Planet).  
Planet = mars.  
  
[1] 30 ?- find_position(earth, Position).  
Position = 3.  
  
[1] 31 ?- find_planet(4, Planet).  
Planet = mars.  
  
[1] 32 ?- planet(Planet, Position).  
Planet = mercury,  
Position = 1 ;  
Planet = venus,  
Position = 2 ;  
Planet = earth,  
Position = 3 ;  
Planet = mars,  
Position = 4 ;  
Planet = jupiter,  
Position = 5 ;  
Planet = saturn,  
Position = 6 ;  
Planet = uranus,  
Position = 7 ;  
Planet = neptune,  
Position = 8.
```